Calculadora de Edad: Validación de Fechas y Cálculo de Edad Exacta

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 1 — De revisión integral (Python & C++) **Fecha:** 2026

Introducción

El objetivo de esta actividad es repasar los conceptos de lógica y algoritmos que se trabajaron el año anterior, pero implementado en **Python** y en **C++** (C++ como lenguaje estructurado de bajo nivel), para poder comparar cómo se resuelve el mismo problema en ambos lenguajes.

El problema que se resolvió fue el cálculo de la **edad exacta** de una persona a partir de su fecha de nacimiento. La palabra clave es *exacta*: no alcanza con restar el año de nacimiento del año actual, porque eso overestimate la edad en un año para quienes todavía no cumplieron años en la fecha de referencia.

Además, el programa no puede accepting cualquier dato: debe verificar que la fecha ingresada **exista realmente en el calendario**, es decir, que respete la cantidad de días de cada mes y la regla de los años bisiestos. Sin ese filtro de consistencia, una fecha como el 31 de abril o el 29 de febrero de 2025 produciría un resultado sin sentido.

Para darle un valor práctico al ejercicio, la solución se implementó en dos lenguajes: **C++**, que permite entender el flujo de datos de bajo nivel con funciones explícitas y tipos declarados, y **Python**, donde la misma lógica se expresa con menos líneas gracias a su sintaxis más declarativa.

Desarrollo

Estructura de la solución

Ambos programas se organizan igual, separando responsabilidades en funciones o módulos con un nombre claro:

1. `esBisiesto(anio)` / `es_bisiesto(anio)` — responde una sola pregunta: ¿es
ese año bisiesto?

2. `esFechaValida(d, m, a)` / `es_fecha_valida(d, m, a)` — aplica el filtro de
consistencia y devuelve si la fecha existe en el calendario.

3. `main()` — pide los datos, invoca la validación y calcula la edad.

La regla del año bisiesto

La condición implementada en ambos lenguajes es la del calendario gregoriano:

```
(anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)
```

Un año es bisiesto si es múltiplo de 4, **pero no** de 100, salvo que además sea múltiplo de 400. Esta regla aparentemente redundante existe porque el calendario gregoriano corrige un error del calendario juliano: cada 100 años se perdían 3 días y el calendario se atrasaba respecto de las estaciones. La solución fue suprimir tres de cada cuatro años bisiestos, salvo los años seculares (los divisibles por 100), que solo cuentan como bisiestos si además son divisibles por 400.

Ejemplos: 2024 **es** bisiesto (múltiplo de 4). 1900 **no** lo es (múltiplo de 4 y de 100, pero no de 400). 2000 **sí** lo es (múltiplo de 4, 100 y 400).

El filtro de consistencia

`esFechaValida` aplica los filtros en orden, de lo más general a lo más específico, y descarta los casos inválidos cuanto antes:

• El año debe estar entre **1900 y 2026**.

• El mes debe estar entre **1 y 12**.

• El día debe estar entre **1 y 31**.

• Abril (4), junio (6), septiembre (9) y noviembre (11) tienen **30 días**: un

día 31 en esos meses se rechaza.

• **Febrero** es el caso especial: admite hasta el 29 si el año es bisiesto y

solo hasta el 28 si no lo es.

Cada filtro tiene un `return false` temprano, lo que en C++ además mejora el rendimiento porque las comprobaciones restantes ni siquiera se ejecutan.

El cálculo de la edad exacta

Este es el punto central de la actividad. La edad se calcula primero como la diferencia simple:

```
edad = anioActual - anioNacimiento
```

y después se ajusta **restándole 1** cuando la persona todavía no cumplió años respecto de la fecha de referencia (8 de julio de 2026):

```
if (mesNacimiento > mesActual)                  -> edad - 1
if (mesNacimiento == mesActual && diaNacimiento > diaActual) -> edad - 1
```

Se evalúan **las dos condiciones juntas** porque una sola no alcanza. Si solo se mirara el mes, alguien nacido el 10 de julio de 2008 vería su edad reducida a pesar de que su cumpleaños ya pasó. Por eso el caso del mismo mes exige comparar también el día. La condición combinada queda:

```
if (mesN > mesA || (mesN == mesA && diaN > diaA))
```

Este detalle es justamente lo que distingue una edad *calculada* de una edad *exacta*.

Diferencias entre C++ y Python

Aspecto C++ Python
Tipo de dato Todo declarado explícitamente (`int`) Inferido por el intérprete

Valor de retorno
`return true;` / `return false;`
`return True` / `return False`

Captura de datos `cin >> d >> m >> a;` (puede fallar silenciosamente) `int(input(...))` dentro de un `try/except ValueError`
Manejo de error `return 1;` como código de salida del sistema `print("[ERROR]...")` y no imprimir la edad

La diferencia más importante desde el punto de vista de la experiencia del usuario es el **manejo de la entrada**: en Python, el bloque `try/except ValueError` convierte un error de tipeo en un mensaje claro, mientras que en C++ el filtro de consistencia solo detecta fechas *imposibles*, no datos que no sean numéricos. Esa es una limitación del lenguaje de bajo nivel que queda visible al comparar las dos versiones.

Conclusión

La actividad cumplió el objetivo de repasar lógica condicional y estructuras de control en dos lenguajes de paradigmática distinta. La conclusión más relevante no es que el programa funcione, sino que **el filtro de entrada es tan importante como el cálculo**: un algoritmo correcto alimentado con datos imposibles produce resultados imposibles.

Comparar C++ y Python sobre el mismo problema dejó una idea clara: el lenguaje de alto nivel no reemplaza al de bajo nivel, lo complementa. C++ obliga a declarar los tipos, a escribir el prototipo de la función y a pensar en qué casillero de memoria vive cada variable; Python permite concentrarse en la lógica. Comprender el primero hace mucho más legible el segundo.

Referencias

• C++ Foundation. (2024). *Basic concepts: Variables, types, and functions*. C++ Team.
<https://isocpp.org/wiki/index.php/Basic_concepts>

• IBM. (2024). *Operators in C++*. IBM Documentation.
<https://www.ibm.com/docs/en/i/24.1.2?topic=c-operators-in-c>

• Python Software Foundation. (2024). *An informal introduction to Python*.
Python Documentation. <https://docs.python.org/3/tutorial/introduction.html>

• Python Software Foundation. (2024). *Built-in functions: `input()`*. Python Documentation.
<https://docs.python.org/3/library/functions.html#input>

• Microsoft. (2024). *Lexical constants*. Microsoft Learn.

<https://learn.microsoft.com/es-es/cpp/cpp/numeric-constants